

Encoder-Decoder Complexity & Cross-Attention.

An encoder-decoder model spends its attention budget in three places, each with its own cost in the input and output lengths $T_{\text{in}}, T_{\text{out}}$. We derive the three terms, evaluate them for $T_{\text{in}}=512$, $T_{\text{out}}=128$, and count parameters to show why the decoder is heavier than the encoder — every number computed in full.

THE COST

$$C_{\text{attn}} = \underbrace{O(T_{\text{in}}^2)}_{\text{enc self}} + \underbrace{O(T_{\text{out}}^2)}_{\text{dec self}} + \underbrace{O(T_{\text{out}} T_{\text{in}})}_{\text{cross}}$$

Worked example. $T_{\text{in}}=512$, $T_{\text{out}}=128$: each attention term and its share; plus a T5-base-ish parameter count ($d=768$, $12+12$ layers).

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Three attention costs	2
2	Data flow (TikZ)	3
3	Worked computation: $T_{\text{in}}=512$, $T_{\text{out}}=128$	3
4	Cost across several shapes	3
5	Why the decoder has more parameters	4
6	Cross-attention recap	4
7	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Three attention costs

A decoder-only model has one attention cost: self-attention over its sequence. An *encoder-decoder* splits the work into three distinct attention computations, because the encoder and decoder process sequences of different lengths and the decoder must additionally read the encoder's output.

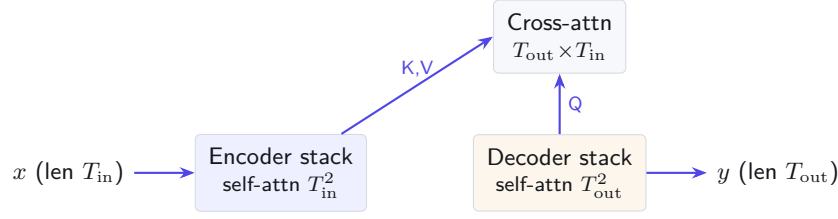
$$C_{\text{attn}} = \underbrace{O(T_{\text{in}}^2)}_{\text{encoder self-attn}} + \underbrace{O(T_{\text{out}}^2)}_{\text{decoder masked self-attn}} + \underbrace{O(T_{\text{out}} T_{\text{in}})}_{\text{cross-attn}} \quad (1)$$

Each $O(\cdot)$ counts the number of query-key score entries in the attention matrix — the quadratic part that dominates as sequences grow.

Block	Query from	Key/Value from	Score-matrix size
Encoder self-attn	encoder	encoder	$T_{\text{in}} \times T_{\text{in}}$
Decoder self-attn	decoder	decoder (masked)	$T_{\text{out}} \times T_{\text{out}}$
Cross-attn	decoder	encoder output	$T_{\text{out}} \times T_{\text{in}}$

Self-attention is a square: every position looks at every position in its *own* sequence. Cross-attention is a rectangle: every decoder position (T_{out} rows) looks at every encoder position (T_{in} columns). Three matrices, two squares and one rectangle — that is the whole story.

2 Data flow (TikZ)



Cross-attention takes its **queries from the decoder** and its **keys and values from the (frozen-per-step) encoder output** — the same scaled dot-product attention of Module 0.1, just with Q and K, V drawn from different stacks.

3 Worked computation: $T_{\text{in}}=512$, $T_{\text{out}}=128$

Evaluate each term in Eq. (1):

$$\begin{aligned}\text{encoder self-attn} &= T_{\text{in}}^2 = 512^2 = 262,144, \\ \text{decoder self-attn} &= T_{\text{out}}^2 = 128^2 = 16,384, \\ \text{cross-attn} &= T_{\text{out}} T_{\text{in}} = 128 \times 512 = 65,536.\end{aligned}$$

The total number of score entries is

$$C_{\text{attn}} = 262,144 + 16,384 + 65,536 = 344,064.$$

Proportions:

$$\frac{262,144}{344,064} = 76.2\%, \quad \frac{16,384}{344,064} = 4.8\%, \quad \frac{65,536}{344,064} = 19.0\%.$$

	enc self	dec self	cross
share	76.2%	4.8%	19.0%

When the input is much longer than the output (summarization, classification of long documents), the encoder self-attention term dwarfs everything else.

4 Cost across several shapes

T_{in}	T_{out}	enc self	dec self	cross	total
128	128	16,384 (33%)	16,384 (33%)	16,384 (33%)	49,152
512	128	262,144 (76%)	16,384 (5%)	65,536 (19%)	344,064
512	512	262,144 (33%)	262,144 (33%)	262,144 (33%)	786,432
1024	128	1,048,576 (88%)	16,384 (1%)	131,072 (11%)	1,196,032
2048	256	4,194,304 (88%)	65,536 (1%)	524,288 (11%)	4,784,128

When $T_{\text{in}} = T_{\text{out}}$ the three terms are *equal* (each one third). As the input grows relative to the output, cost migrates into the *encoder self-attention* (T_{in}^2) and, secondarily, the *cross term* ($T_{\text{out}}T_{\text{in}}$); the decoder self-attention becomes negligible.

5 Why the decoder has more parameters

Per layer, with model width d and feed-forward width $d_{\text{ff}} = 4d$ (T5-base):

- Multi-head attention block = $4d^2$ (the W_Q, W_K, W_V, W_O projections, each $d \times d$).
- Feed-forward block = $2(d \cdot d_{\text{ff}}) = 2 \cdot 4d^2 = 8d^2$.

So an **encoder layer** has one attention block plus FFN:

$$P_{\text{enc-layer}} = 4d^2 + 8d^2 = 12d^2,$$

while a **decoder layer** adds a *second* attention block (cross-attention):

$$P_{\text{dec-layer}} = \underbrace{4d^2}_{\text{self}} + \underbrace{4d^2}_{\text{cross}} + \underbrace{8d^2}_{\text{FFN}} = 16d^2.$$

The decoder layer is $\frac{16}{12} = 1.33\times$ the encoder layer.

T5-base-ish totals. With $d = 768$ (so $d^2 = 589,824$) and $L = 12$ layers each side:

$$P_{\text{enc}} = 12 \times (12d^2) = 12 \times 7,077,888 = 84,934,656 \approx \mathbf{84.9 \text{ M}},$$

$$P_{\text{dec}} = 12 \times (16d^2) = 12 \times 9,437,184 = 113,246,208 \approx \mathbf{113.2 \text{ M}}.$$

Adding the shared token embedding $|V| \cdot d = 32,128 \times 768 = 24,674,304 \approx 24.7 \text{ M}$, the stacks plus embedding total $84.9 + 113.2 + 24.7 \approx \mathbf{222.9 \text{ M}}$, in the right neighbourhood of T5-base's $\sim 220 \text{ M}$.

Component	d^2 multiple	params ($d=768, L=12$)
Encoder layer	$12d^2$	7.08 M
Decoder layer	$16d^2$	9.44 M
Encoder (12)	—	84.9 M
Decoder (12)	—	113.2 M
Shared embedding	—	24.7 M
Total	—	$\approx 222.9 \text{ M}$

6 Cross-attention recap

Cross-attention is exactly the Module 0.1 operation with its inputs split across stacks. Let $H_{\text{enc}} \in \mathbb{R}^{T_{\text{in}} \times d}$ be the encoder output and $H_{\text{dec}} \in \mathbb{R}^{T_{\text{out}} \times d}$ the current decoder states:

$$Q = H_{\text{dec}}W_Q, \quad K = H_{\text{enc}}W_K, \quad V = H_{\text{enc}}W_V,$$

$$\text{CrossAttn} = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{T_{\text{out}} \times d}.$$

The score matrix $QK^\top$ is $T_{\text{out}} \times T_{\text{in}}$ — the rectangular cross term of Eq. (1). The encoder runs *once*; K, V are computed once and reused at every decode step.

For long inputs it is tempting to budget for the decoder, but the **encoder self-attention** (T_{in}^2) **and the cross term** ($T_{\text{out}}T_{\text{in}}$) **dominate**. At $T_{\text{in}}=1024$, $T_{\text{out}}=128$ they are 88% and 11% of cost while decoder self-attention is barely 1%. Optimizing the decoder's masked self-attention there buys almost nothing; cap or sparsify the *input* length instead.

7 Where this sits in the track

Module	Connection
0.1 Attention	cross-attention is scaled dot-product attention, Q vs. K, V split across stacks
4.1 T1	supplies the (x, y) pairs whose lengths $T_{\text{in}}, T_{\text{out}}$ drive this cost
4.1 T2 (here)	the compute and parameter budget of the full encoder-decoder

A Reproducible (NumPy)

Inputs: $T_{\text{in}}=512$, $T_{\text{out}}=128$; $d=768$, $L=12$, $|V|=32128$.

```
import numpy as np
# --- three attention terms ---
def terms(Ti, To): return Ti**2, To**2, To*Ti, Ti**2+To**2+To*Ti
e,dpos,c,tot = terms(512,128)
print(e, dpos, c, tot) # 262144 16384 65536 344064
print([round(x/tot*100,1) for x in (e,dpos,c)]) # [76.2, 4.8, 19.0]
# --- parameter counts ---
d = 768; L = 12; V = 32128
enc_layer, dec_layer = 12*d**2, 16*d**2
P_enc, P_dec = L*enc_layer, L*dec_layer
emb = V*d
print(P_enc/1e6, P_dec/1e6, emb/1e6) # 84.93 113.25 24.67
print((P_enc+P_dec+emb)/1e6) # 222.86
print(dec_layer/enc_layer) # 1.3333
```